HookFlow

Product Requirements Document (PRD)

Version: 1.0
Status: Draft
Product Type: Developer Infrastructure / Backend SaaS
Primary Objective: Build a reliable, secure, and usable webhook delivery platform.

---

1. Executive Summary

HookFlow is a developer-facing platform that enables applications to send events to external HTTP endpoints reliably.

Developers integrating webhooks into their applications often need to implement delivery queues, retry policies, request signing, failure handling, event logging, and monitoring. Building these capabilities independently consumes engineering time and creates opportunities for reliability and security problems.

HookFlow provides these capabilities through a unified API and dashboard.

Developers can register webhook endpoints, publish events, monitor delivery attempts, configure retry behavior, and replay failed deliveries. HookFlow processes deliveries asynchronously and maintains a persistent record of each event's delivery history.

The platform will initially target independent developers, small SaaS companies, and engineering teams that need webhook infrastructure without maintaining their own delivery systems.

1.1 Product Vision

Make reliable webhook delivery accessible to developers through a simple API and a transparent, observable delivery pipeline.

1.2 Core Value Proposition

Send events once to HookFlow; let HookFlow manage delivery, retries, and observability.

The platform will not promise that every destination processes every event exactly once. Instead, it will provide durable event records, at-least-once delivery attempts under documented operating assumptions, duplicate-aware processing, and transparent delivery status.

---

2. Problem Statement

Applications frequently need to notify other applications when important events occur.

Examples include:

- An order is created.
- A payment succeeds.
- A document finishes processing.
- A user account is created.
- A subscription expires.

A basic implementation sends an HTTP request directly to the receiving application. However, the destination might be temporarily unavailable, respond slowly, return an error, or accept a request that the sender subsequently fails to acknowledge internally.

Developers must then solve several problems.

Problem	Consequence
Destination unavailable	Notifications fail
No retry mechanism	Temporary failures become permanent
Duplicate requests	Business operations may execute multiple times
Missing delivery history	Developers cannot diagnose failures
No request authentication	Receivers cannot reliably verify authenticity
Blocking HTTP requests	Application latency increases
No replay functionality	Recovery requires manual intervention
No monitoring	Persistent delivery failures go unnoticed

HookFlow addresses these problems through a centralized webhook delivery service.

3. Product Goals and Non-Goals

3.1 Goals

The initial product must:

1. Allow users to create accounts and manage API credentials.
2. Allow users to register and configure webhook endpoints.
3. Accept event submissions through an authenticated REST API.
4. Persist events before acknowledging successful ingestion.
5. Deliver events asynchronously to registered destinations.
6. Retry eligible failed deliveries using configurable policies.
7. Sign outgoing requests so receivers can verify authenticity.
8. Maintain detailed delivery-attempt history.
9. Allow users to inspect and replay failed deliveries.
10. Provide a dashboard for managing endpoints, events, and deliveries.
11. Enforce tenant isolation, authentication, and outbound-request security.
12. Provide automated tests, API documentation, monitoring, and deployment configuration.

3.2 Non-Goals for the Initial Release

The initial release will not attempt to provide:

- A general-purpose event streaming platform.
- Guaranteed exactly-once delivery or processing.
- Kafka-based distributed streaming.
- A Kubernetes-based deployment architecture.
- Arbitrary workflow orchestration.
- A visual workflow builder.
- A marketplace of third-party integrations.
- A mobile application.
- Complex enterprise billing and subscription management.
- Guaranteed global delivery or multi-region disaster recovery.

These features may be considered after validating the initial product.

4. Target Users and Personas

4.1 Independent Developer

Profile: A developer building a personal project, API, or small SaaS application.

Needs:

- A straightforward API.
- Simple endpoint registration.
- Clear documentation.
- Visibility into delivery failures.
- Minimal infrastructure maintenance.

Success scenario: The developer integrates HookFlow into an application and can inspect the delivery history of every submitted event.

4.2 Startup Backend Engineer

Profile: An engineer maintaining multiple services that communicate through HTTP events.

Needs:

- Reliable asynchronous delivery.
- Configurable retries.
- Secure webhook signatures.
- Event replay.
- API-based endpoint management.
- Monitoring and debugging tools.

Success scenario: The engineer can recover from a temporary downstream outage without implementing a custom retry system.

4.3 Small SaaS Team

Profile: A small engineering team offering integrations to customers.

Needs:

- Multiple endpoints.
- Event subscriptions.
- Team access in a future release.
- Usage visibility.

- Delivery logs and incident diagnosis.

Success scenario: The team uses HookFlow to provide outbound webhooks to its own customers.

---

5. Product Scope

The product will consist of four primary components.

5.1 Developer API

Responsible for:

- Authenticating API requests.
- Managing endpoints.
- Accepting event submissions.
- Returning event identifiers.
- Exposing event and delivery status.
- Supporting delivery replay.

5.2 Delivery Engine

Responsible for:

- Scheduling deliveries.
- Executing outbound HTTP requests.
- Signing request payloads.
- Applying timeouts.
- Recording attempt outcomes.
- Scheduling retries.
- Recovering abandoned work.

5.3 Web Dashboard

Responsible for:

- Account and API key management.
- Endpoint configuration.
- Event browsing.
- Delivery inspection.
- Failure investigation.
- Manual replay.

- Basic usage analytics.

5.4 Background Processing and Observability

Responsible for:

- Queue management.
- Worker coordination.
- Delivery scheduling.
- Metrics collection.
- Structured logging.
- Operational health monitoring.

---

6. Functional Requirements

Priority definitions:

- **P0:** Required for the initial usable release.
- **P1:** Required for a robust public beta.
- **P2:** Suitable for a later release.

FR-01: User Authentication

Priority: P0

Users must be able to create accounts and authenticate securely.

Requirements:

- Support account registration and login.
- Store passwords using a suitable password-hashing algorithm.
- Support logout and session invalidation.
- Provide account-level data isolation.
- Protect authenticated dashboard routes.
- Apply request limits to authentication endpoints.

Acceptance criteria:

- Unauthenticated users cannot access protected resources.
- Users cannot retrieve another user's endpoints or events.
- Passwords are never stored in plaintext.

- Invalid credentials produce an appropriate authentication error.

FR-02: API Key Management

Priority: P0

Users must be able to authenticate programmatic requests using API keys.

Requirements:

- Generate API keys securely.
- Display the complete key only when initially created.
- Store a secure key verifier rather than the plaintext key.
- Support revocation.
- Associate each key with its owner.
- Allow users to identify keys using a non-secret prefix.

Acceptance criteria:

- A valid key authorizes requests according to its permissions.
- An invalid or revoked key cannot authorize new requests.
- API keys are never written to application logs.
- API key ownership is enforced on every authenticated operation.

FR-03: Endpoint Management

Priority: P0

Users must be able to register webhook destinations.

Requirements:

- Create an endpoint with a destination URL.
- Configure event subscriptions.
- Enable or disable an endpoint.
- View endpoint details.
- Update endpoint configuration.
- Delete or archive an endpoint.
- Generate and rotate signing secrets.

Acceptance criteria:

- An endpoint belongs to exactly one customer account.
- Only its authorized owner can modify its configuration.
- Disabled endpoints receive no new deliveries.

- Existing delivery history remains available according to the retention policy.

FR-04: Event Ingestion

Priority: P0

Applications must be able to publish events through a REST API.

Example request:

```
POST /v1/events
```

```
None
{
  "event_type": "order.created",
  "data": {
    "order_id": "ORD-101",
    "amount": 2499
  }
}
```

Requirements:

- Authenticate the request.
- Validate the event type and JSON payload.
- Generate a unique event identifier.
- Persist the event and associated delivery work.
- Identify eligible subscribed endpoints.
- Return the event ID and processing status.
- Support idempotency keys.
- Enforce payload-size limits.

Example successful response:

```
None
{
  "event_id": "evt_12345",
  "status": "accepted",
  "created_at": "2026-10-30T12:00:00Z"
}
```

Acceptance criteria:

- A successful response is returned only after the event and its required delivery records have been durably persisted.
- The API does not wait for destination endpoints to respond.
- Repeating a request with the same idempotency key and equivalent parameters does not create another event.
- Reusing an idempotency key with a different payload returns a conflict.
- Events are delivered only to eligible endpoints belonging to the submitting account.

FR-05: Asynchronous Delivery

Priority: P0

HookFlow must deliver events independently of the ingestion request.

Requirements:

- Queue delivery work.
- Execute outbound HTTP POST requests.
- Include the event ID, event type, timestamp, and event data.
- Apply configurable connection and response timeouts.
- Record the result of every attempt.
- Support concurrent workers.
- Prevent simultaneous workers from independently claiming the same active delivery.

Acceptance criteria:

- A successful destination response marks the delivery as delivered.
- An unsuccessful attempt is recorded.
- One endpoint's failure does not prevent delivery to other endpoints.
- Worker termination does not permanently lose durably recorded delivery work.
- Recovery logic can identify and reclaim abandoned processing jobs safely.

FR-06: Automatic Retries

Priority: P0

HookFlow must retry eligible failed deliveries.

Requirements:

- Configure maximum attempts.
- Support exponential backoff.
- Add randomized jitter to retry delays.

- Distinguish retryable from non-retryable responses.
- Persist the next scheduled attempt.
- Stop automatic retries when the configured limit is reached.

Default policy for the initial release:

- Maximum attempts: 5, including the initial attempt.
- Retryable responses: HTTP 429, selected HTTP 5xx responses, connection failures, and timeouts.
- Other HTTP 4xx responses: generally non-retryable.
- Retry delays: configurable, with exponential backoff and jitter.

Acceptance criteria:

- Retry scheduling survives application restarts.
- A delivery is not retried before its scheduled time.
- The attempt count is accurate.
- Exhausted deliveries become eligible for manual replay.
- Operators can distinguish permanent failures from scheduled retries.

FR-07: Request Signing

Priority: P0

Outgoing webhook requests must be verifiable by receiving applications.

Requirements:

- Use HMAC-SHA256.
- Include a request timestamp.
- Sign the timestamp and exact serialized request body.
- Provide the signature in a dedicated HTTP header.
- Support secret rotation.
- Document receiver-side verification.

Acceptance criteria:

- The receiver can independently verify a valid signature.
- Changing the signed body invalidates the signature.
- Invalid signatures can be detected.
- Secret values are not exposed in delivery logs.
- Documentation explains timestamp validation and replay protection.

FR-08: Delivery History

Priority: P0

Users must be able to inspect delivery attempts.

Requirements:

- Show event ID and endpoint.
- Show delivery status.
- Show attempt count.
- Record HTTP response status.
- Record attempt start and completion timestamps.
- Record response latency.
- Store sanitized error details.
- Show the next scheduled retry, when applicable.

Acceptance criteria:

- Every completed attempt has a corresponding history record.
- Failed attempts remain visible after a later successful retry.
- Users cannot view another customer's delivery records.
- Response-body storage is bounded and sensitive information is handled safely.

FR-09: Manual Replay

Priority: P0

Users must be able to request a new delivery attempt for an eligible failed delivery.

Requirements:

- Provide a replay action in the dashboard.
- Provide an authenticated replay API.
- Preserve the original event ID.
- Maintain a history of replay operations.
- Apply current endpoint security and delivery limits.
- Prevent unauthorized replay operations.

Acceptance criteria:

- Replay creates an auditable delivery operation.
- The original attempt history remains intact.
- Replay does not silently create a new business event.
- Repeated replay requests cannot bypass rate limits or cause uncontrolled delivery storms.

FR-10: Dashboard and Analytics

Priority: P0

The dashboard must provide the core workflows needed to use the product.

Requirements:

- Display total events and deliveries.
- Show successful, pending, retrying, and failed deliveries.
- List and filter events.
- List and configure endpoints.
- Inspect delivery history.
- Trigger eligible replays.
- Display basic time-range-based metrics.

Acceptance criteria:

- Users can navigate from an event to its deliveries.
- Users can identify why a delivery failed.
- Filters work across relevant statuses and dates.
- Dashboard data is scoped to the authenticated account.

FR-11: Rate Limiting and Quotas

Priority: P1

Requirements:

- Apply limits to event ingestion.
- Apply limits to authentication endpoints.
- Configure per-account quotas.
- Limit outbound delivery concurrency.
- Return HTTP 429 when applicable.
- Expose appropriate rate-limit information.

FR-12: Notifications

Priority: P1

Requirements:

- Allow users to enable delivery-failure notifications.
- Notify users when configured failure thresholds are reached.

- Avoid repeated notifications for the same incident.
- Track notification failures separately from webhook delivery failures.

FR-13: Operational Monitoring

Priority: P1

Requirements:

- Expose API latency and error metrics.
- Track queue depth.
- Track worker health.
- Measure delivery latency and success rates.
- Emit structured logs.
- Provide health-check endpoints.
- Alert operators to persistent processing failures.

---

7. Non-Functional Requirements

7.1 Reliability

- Persist accepted events and required delivery records before acknowledging ingestion.
- Recover unfinished work after worker crashes.
- Make queue publication recoverable.
- Ensure that delivery state transitions are concurrency-safe.
- Document at-least-once delivery semantics.
- Avoid claiming exactly-once delivery.

7.2 Performance

Initial engineering targets, to be validated through load testing:

Metric	Initial target
Event ingestion latency	p95 below 200 ms under defined test load
Event ingestion response	Independent of destination response time

Dashboard API latency	p95 below 300 ms under defined test load
Delivery concurrency	Configurable
Payload limit	1 MiB by default
Delivery timeout	Configurable; initially 10 seconds

These are development targets, not guarantees. Results must be measured against a documented deployment configuration and workload.

7.3 Security

- Enforce authentication and authorization.
- Isolate tenant data.
- Encrypt traffic in transit.
- Protect stored credentials and signing secrets.
- Validate incoming payloads.
- Protect against SSRF and DNS rebinding.
- Prevent unauthorized access to internal network resources.
- Limit response-body storage and payload sizes.
- Redact sensitive data from logs.
- Apply request and concurrency limits.

7.4 Scalability

The architecture should allow API instances and delivery workers to scale independently.

The initial deployment does not require distributed multi-region infrastructure.

7.5 Maintainability

- Use modular application components.
- Provide versioned REST APIs.
- Manage schema changes with database migrations.
- Maintain automated tests.
- Document environment configuration.
- Use structured logging and consistent error responses.

7.6 Observability

The system must expose sufficient information to investigate:

- Missing deliveries.
- Repeated timeouts.
- Growing queues.
- Worker failures.
- Elevated API latency.
- Persistent endpoint outages.

---

8. Proposed API Specification

All endpoints will use the `/v1` prefix.

Method	Endpoint	Purpose
POST	`/v1/events`	Publish an event
GET	`/v1/events`	List events
GET	`/v1/events/{event_id}`	Retrieve an event
GET	`/v1/events/{event_id}/deliveries`	List event deliveries
POST	`/v1/events/{event_id}/replay`	Request replay
POST	`/v1/endpoints`	Register an endpoint
GET	`/v1/endpoints`	List endpoints
GET	`/v1/endpoints/{endpoint_id}`	Retrieve endpoint details
PATCH	`/v1/endpoints/{endpoint_id}`	Update endpoint configuration
DELETE	`/v1/endpoints/{endpoint_id}`	Delete or archive an endpoint
POST	`/v1/endpoints/{endpoint_id}/rotate-secret`	Rotate signing secret
GET	`/v1/deliveries/{delivery_id}`	Retrieve delivery status
GET	`/v1/deliveries/{delivery_id}/attempts`	Retrieve attempt history

POST	`/v1/deliveries/{delivery_id}/replay`	Replay a failed delivery
GET	`/health/live`	Check process liveness
GET	`/health/ready`	Check service readiness

The two replay endpoints may be consolidated into one canonical endpoint during API design.

All list endpoints should support pagination. Sensitive operations must enforce ownership and authorization.

The API will use standard HTTP status codes and a consistent JSON error format.

---

9. Data Model

The initial database schema will include the following entities:

1. **Users:** Account information.
2. **API Keys:** Authentication credentials and revocation metadata.
3. **Endpoints:** Destination URLs and endpoint configuration.
4. **Endpoint Subscriptions:** Event types accepted by each endpoint.
5. **Events:** Immutable event payloads and metadata.
6. **Deliveries:** Delivery state for each event-endpoint pair.
7. **Delivery Attempts:** Individual HTTP attempt records.
8. **Outbox Events:** Durable records awaiting publication to the queue.
9. **Audit Logs:** Important user and administrative actions.

Important relationships

- One user can own multiple API keys.
- One user can register multiple endpoints.
- One event can produce multiple deliveries.
- One delivery can have multiple attempts.
- One endpoint can subscribe to multiple event types.
- Each delivery belongs to an event and an endpoint.

Data consistency rules

- Event IDs must be unique.
- Idempotency keys must be unique within their defined customer scope.
- Delivery claims must be atomic.

- Attempt history must be append-oriented.
- Event ownership and endpoint ownership must be validated.
- Durable delivery work must remain recoverable if queue publication fails.

---

10. Delivery State Machine

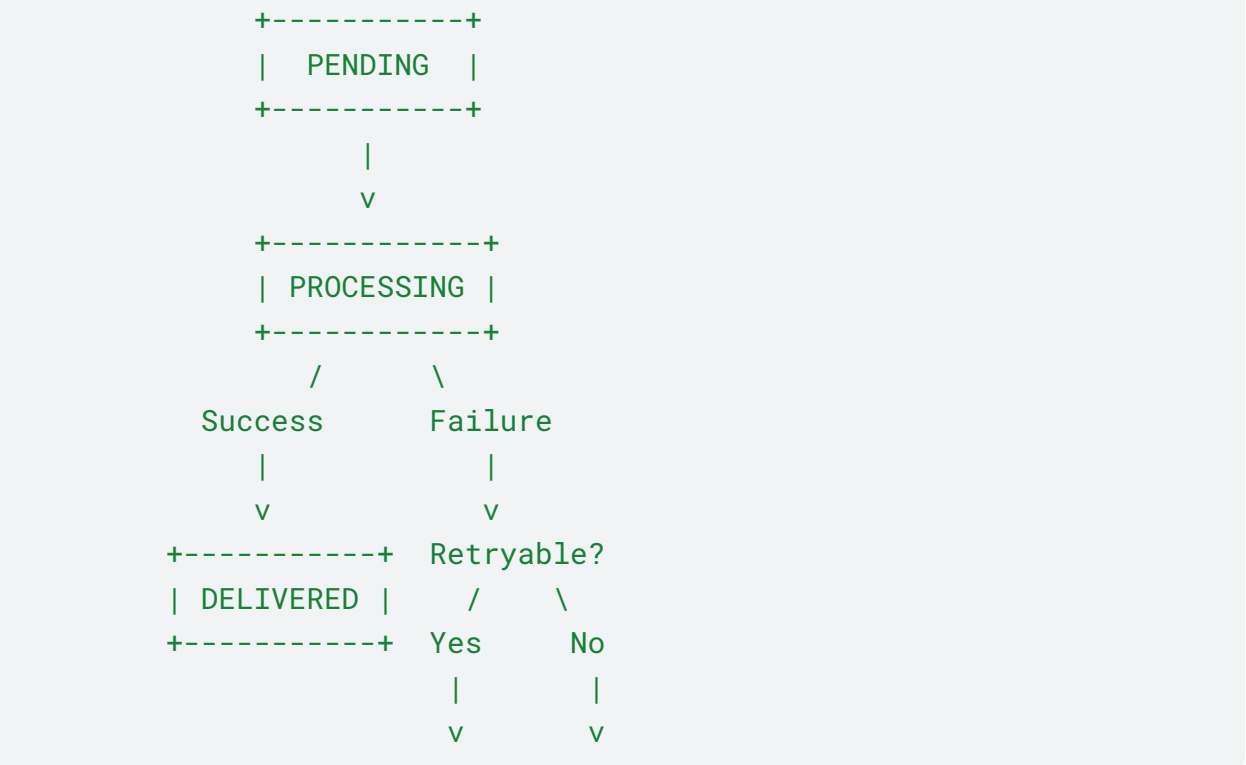
A delivery must follow a well-defined lifecycle.

Proposed states:

- `pending`
- `processing`
- `retrying`
- `delivered`
- `failed`
- `cancelled`

State transitions

None



```
                +---------+ +--------+
                | RETRYING| | FAILED |
                +---------+ +--------+
                     |
                     v
                PROCESSING
```

If a retryable failure exhausts the configured attempt budget, the delivery transitions to `failed`.

A separate cancellation path may apply to pending work, provided the system defines what happens when cancellation races with active processing.

State transitions must be validated and persisted transactionally.

---

11. User Experience Requirements

The product should be usable without requiring users to understand its internal queue architecture.

Onboarding

1. Register an account.
2. Generate an API key.
3. Register a destination endpoint.
4. Configure event subscriptions.
5. Send a test event.
6. Inspect the delivery result.

Endpoint details page

Must display:

- Destination URL.
- Subscribed event types.
- Active or disabled state.
- Signing-secret management.
- Recent deliveries.
- Failure history.

Event details page

Must display:

- Event ID and event type.
- Original payload.
- Creation timestamp.
- Matching endpoints.
- Delivery status for each endpoint.
- Attempt history and response details.
- Replay action when permitted.

Error presentation

Errors should be actionable.

For example, show:

"Delivery failed: destination returned HTTP 503. The next retry is scheduled for 14:35 UTC."

Do not expose internal stack traces, credentials, or sensitive infrastructure details.

---

12. Technical Architecture

Initial technology choices

Layer	Technology
Frontend	Next.js and TypeScript
Backend API	FastAPI
Relational database	PostgreSQL
Background queue	Redis
Worker framework	Celery
ORM	SQLAlchemy
Migrations	Alembic

Testing	Pytest
Containers	Docker Compose
CI/CD	GitHub Actions
Monitoring	Prometheus and Grafana

Reliability architecture

The API will write the event, matching delivery records, and outbox records in one PostgreSQL transaction.

An outbox publisher will subsequently enqueue pending delivery work.

Workers will claim deliveries using atomic database operations or leases, perform the HTTP request, and record the outcome. Failed work will be rescheduled according to its retry policy.

If a worker crashes after the destination receives a request but before success is recorded, the delivery may be attempted again. The platform must preserve the same event ID and document this at-least-once behavior.

Redis will coordinate processing but will not be the sole durable record of accepted work.

---

13. Security and Abuse Prevention

Webhook delivery introduces a critical security concern: the platform makes outbound HTTP requests to user-configured destinations.

The following controls are mandatory before enabling unrestricted public endpoint registration.

13.1 SSRF Prevention

- Restrict destination URL schemes to HTTP and HTTPS.
- Reject loopback, private, link-local, reserved, and otherwise prohibited IP ranges.
- Validate DNS resolution and the destination IP.
- Defend against DNS rebinding and redirect-based bypasses.
- Disable redirects by default.
- Enforce network-level egress restrictions.
- Apply strict timeouts and response-size limits.

13.2 Credential Protection

- Hash API keys for verification.
- Encrypt signing secrets at rest.
- Restrict secret access.
- Redact credentials from logs.
- Support key and secret revocation or rotation.

13.3 Tenant Isolation

Every data-access operation must enforce ownership. Random or guessed resource IDs must not allow access to another tenant's records.

13.4 Resource Protection

- Limit event payload size.
- Limit endpoint count per account.
- Limit ingestion requests.
- Limit concurrent outbound requests.
- Enforce queue and retry quotas.
- Apply retention and cleanup policies.

---

14. Testing Strategy

Unit Testing

Test:

- Request validation.
- Signature generation and verification.
- Retry-delay calculations.
- Idempotency behavior.
- State-transition rules.
- Authorization checks.

Integration Testing

Test:

- Database transactions.
- Event ingestion and outbox publication.
- Queue and worker interactions.
- Endpoint delivery.
- Retry scheduling.
- Secret rotation.
- Replay behavior.

Failure Testing

Simulate:

- HTTP 500 responses.
- HTTP 429 responses.
- Connection timeouts.
- DNS failures.
- Worker termination.
- Database unavailability.
- Redis unavailability.
- Queue publication failures.
- Duplicate submissions.
- A destination processing a request before the worker crashes.

Security Testing

Test:

- Unauthorized resource access.
- Cross-tenant access attempts.
- Forged signatures.
- SSRF attempts.
- Malicious redirect destinations.
- Oversized payloads.
- Rate-limit bypass attempts.
- Credential leakage through logs.

Load Testing

Measure:

- Ingestion throughput.
- p95 and p99 ingestion latency.
- Delivery throughput.

- Queue growth.
- Worker recovery time.
- Database utilization.

Record the test environment and workload alongside every performance result.

---

15. Success Metrics

The product will be evaluated on usability, reliability, and engineering quality.

Product metrics

- Number of registered users.
- Number of configured endpoints.
- Number of events accepted.
- Percentage of users who successfully complete a test delivery.
- Percentage of users who return and publish another event.
- Number of external developers actively using the platform.

Reliability metrics

- Percentage of eligible deliveries eventually delivered within a defined observation window.
- Percentage of deliveries with complete attempt history.
- Number of lost accepted events.
- Queue processing delay.
- Worker recovery time.
- Rate of duplicate delivery attempts.

Engineering quality metrics

- Automated test coverage of critical delivery paths.
- Successful recovery from worker crashes.
- Successful execution of security tests.
- API latency under defined workloads.
- Documented deployment and recovery procedures.

Targets for adoption and reliability should be established after the first usable deployment and initial testing.

---

16. Release Plan

Milestone 1: Core MVP

Target: Week 1–2

Deliver:

- User authentication.
- API key management.
- Endpoint registration.
- Event ingestion.
- PostgreSQL persistence.
- Background delivery.
- Basic delivery history.
- Initial dashboard.

Exit criteria: A developer can register an endpoint, submit an event, receive it, and inspect its delivery result.

Milestone 2: Reliability

Target: Week 3

Deliver:

- Configurable retries.
- Delivery-attempt history.
- HMAC signatures.
- Idempotency.
- Manual replay.
- Durable outbox processing.
- Worker recovery.

Exit criteria: Transient failures can be retried, duplicate ingestion is handled correctly, and unfinished work can be recovered after a worker restart.

Milestone 3: Public Beta Hardening

Target: Week 4–5

Deliver:

- SSRF protections.
- Rate limiting.
- Metrics and structured logging.
- Integration tests.
- Load testing.
- API documentation.
- Production deployment.
- Error handling and retention policies.

Exit criteria: Critical security and recovery tests pass, operational metrics are available, and the system can be used safely within documented limits.

Milestone 4: Product Improvements

Target: Week 6 and beyond

Potential additions:

- Email alerts.
- Team accounts.
- Usage analytics.
- SDKs.
- Endpoint health indicators.
- Advanced retry configuration.
- Customer quotas.

These are secondary to the reliability of the core delivery pipeline.

---

17. Risks and Mitigations

Risk	Mitigation
Queue loses or delays work	Durable outbox and database-driven recovery
Worker crashes during delivery	Leases, recovery, and at-least-once processing
Duplicate delivery	Stable event IDs and receiver-side idempotency guidance

SSRF attack	Destination validation, redirect controls, and egress restrictions
Unbounded retries	Maximum attempts, quotas, and backoff
Database growth	Retention policies, indexing, and archival
Slow destinations	Strict timeouts and bounded concurrency
Credential leakage	Secret hashing or encryption, redaction, and rotation
Scope expansion	Release-based prioritization
No real users	Early deployment, developer feedback, and usable documentation

---

18. Assumptions and Open Questions

The initial PRD assumes:

1. HookFlow is a multi-tenant SaaS application.
2. The first release supports outbound HTTP webhooks.
3. Events use JSON payloads.
4. PostgreSQL is the durable system of record.
5. Redis is used for background processing coordination.
6. The initial deployment uses a single region.
7. Delivery semantics are at-least-once, not exactly-once.
8. The first version is operated by one developer.
9. A basic dashboard accompanies the API.
10. Advanced enterprise features are deferred.

The following decisions should be revisited before implementation:

- Should event subscriptions use exact event-type matching only, or support wildcard patterns?
- How long should events and delivery attempts be retained?
- Should endpoint deletion disable new deliveries immediately while preserving existing history?
- Should users be able to customize retry policies in the MVP?
- Which deployment provider and object-storage options will be used?
- Will the initial service be public, private, or limited to invited beta users?

---

19. Definition of Done

HookFlow's first public beta is complete when:

- Users can register and authenticate.
- API keys can be created and revoked.
- Users can register secure webhook endpoints.
- Events can be accepted and persisted.
- Matching delivery work can be recovered after queue or worker failures.
- Workers can deliver events asynchronously.
- Retry policies behave as documented.
- Requests include verifiable signatures.
- Idempotency is enforced for event ingestion.
- Delivery attempts are durably recorded.
- Failed deliveries can be inspected and replayed.
- Tenant isolation tests pass.
- SSRF protections are implemented and tested.
- Critical workflows have automated integration tests.
- The application is deployable through documented configuration.
- Health metrics and operational logs are available.
- API documentation enables a new developer to complete a test delivery without assistance.

---

20. Final Product Direction

HookFlow should begin as a focused webhook delivery service rather than a general-purpose distributed event platform.

Its first job is to reliably accept an event, persist the work, deliver the event to subscribed endpoints, and explain exactly what happened.

The core product loop is:

Publish event → Persist event → Schedule delivery → Deliver → Record result → Retry if necessary → Inspect or replay.

Every advanced feature should strengthen that loop.

The initial release should prioritize correctness, security, transparency, and ease of integration over the number of technologies used or the complexity of the architecture.

The product is successful when a developer can integrate HookFlow into a real application, trust its documented delivery behavior, and diagnose failures without needing to inspect the underlying infrastructure.